STUDENT MANAGEMENT SYSTEM

A Python Programming Mini Project

Submitted by: ____Enumula Abhilash Reddy_____
Roll Number: ____26BAC10014_____
Programme: B.Tech — First Year
Course: Introduction to Problem Solving and Programming
Department: _____ECE(AI & Cybernetics)_____
Academic Year: _____2026-2030_____

Abstract

The Student Management System is a beginner-friendly, menu-driven application developed using Python. It helps maintain student records and perform basic academic calculations. The program supports adding, viewing, searching and deleting student records, calculating class averages, identifying the topper and sorting students by percentage. Student data is stored in a dictionary during execution and saved to a JSON file so that it can be loaded again in a later run.

1. Introduction

Maintaining student information manually can be repetitive and may lead to calculation or record-keeping errors. This project demonstrates how fundamental programming concepts can be combined to create a small, useful application. It is designed for a first-year engineering student and uses core Python features rather than a database or graphical interface.

2. Objectives

- Create and manage student records using roll numbers.
- Accept marks for five subjects and validate the entered values.
- Calculate total marks, percentage and grade.
- Search, display and delete records.
- Calculate the class average and identify the topper.
- Sort students by percentage and preserve records using JSON file handling.

3. Tools and Technologies

- Language: Python 3
- Editor: Visual Studio Code (or any Python editor)
- Storage: JSON file (students.json)
- Libraries: json and os (Python standard library)
- Interface: Command-line / terminal

4. System Design

The program displays a menu and accepts the user's choice. Each menu option calls a separate function. Student records are stored in a dictionary using the roll number as the key. Each value contains the student's name and a list of marks in the fixed subject order: Python, Maths, Physics, English and Computer.
Record structure (conceptual): students[roll] = { name: student_name, marks: [m1, m2, m3, m4, m5] }

5. Functional Modules

Add Student

Accepts a unique roll number, name and five marks. Marks are checked to be numeric and within 0–100.

View All Students

Iterates through the dictionary and displays each student's marks, total, percentage and grade.

Search Student

Looks up a record using its roll number.

Class Average

Computes each student's percentage and averages those percentages.

Find Topper

Scans the records and retains the student with the highest percentage.

Delete Student

Removes the selected record and saves the updated data.

Sort Students

Displays a descending ranking based on percentage.

Save and Load

Writes records to students.json and loads them when the program starts.

6. Algorithm

1. Start the program and load saved records from students.json if available.
1. Display the menu repeatedly until the Exit option is selected.
1. Read the user's choice.
1. Run the corresponding function: add, view, search, average, topper, delete or sort.
1. Save changes after adding or deleting a student.
1. On Exit, stop the loop and terminate the program.

7. Sample Outputs

The following are illustrative terminal-output mockups showing the expected appearance and example data. They are not screenshots captured from the student's computer; replace them with actual screenshots after running and testing the program.

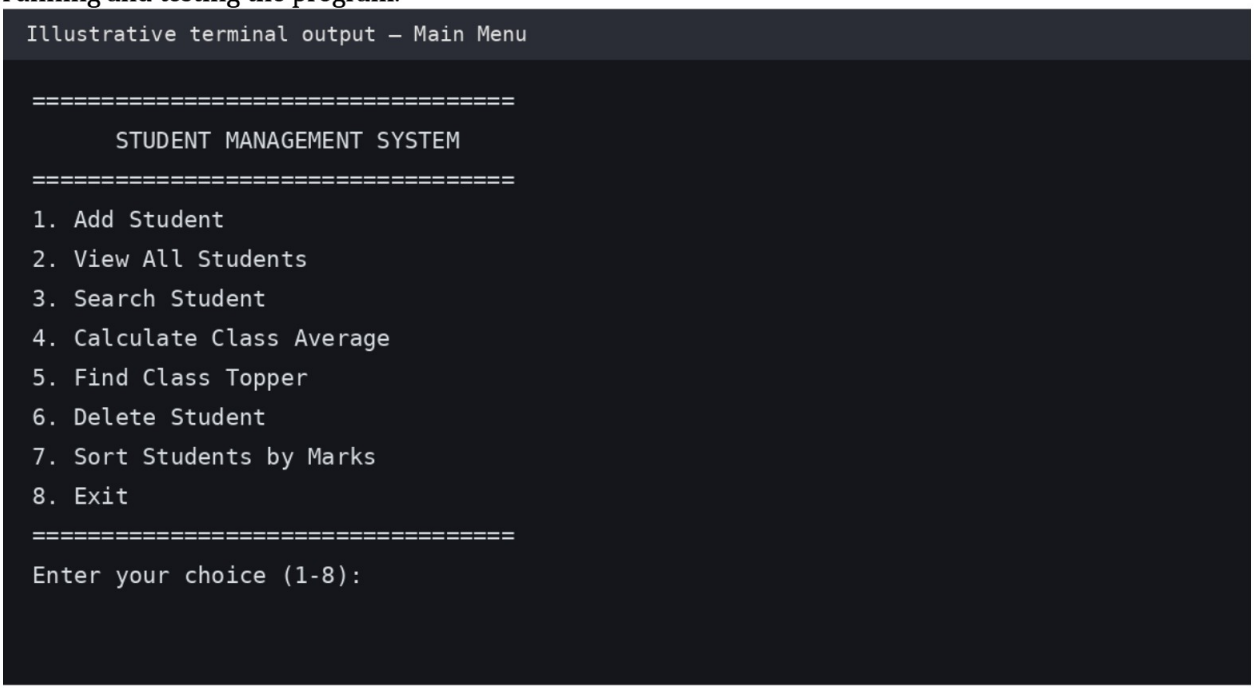


Figure 1. Main menu

```
Illustrative terminal output — Add and View

===== ADD STUDENT =====
Enter roll number: 101
Enter student name: Akshaya
Enter Python marks: 95
Enter Maths marks: 88
Enter Physics marks: 91
Enter English marks: 85
Enter Computer marks: 96
Student added successfully!

===== ALL STUDENTS =====
Roll Number: 101
Name: Akshaya
Python: 95 | Maths: 88 | Physics: 91
English: 85 | Computer: 96
```

Figure 2. Adding a student and viewing the record

```
Illustrative terminal output — Average and Ranking

===== CLASS AVERAGE =====
Number of students: 2
Class average: 84.5 %

===== RANK LIST =====
1. Akshaya | Roll: 101 | Percentage: 91.0 %
2. Rahul   | Roll: 102 | Percentage: 78.0 %

===== CLASS TOPPER =====
Roll Number: 101
Name: Akshaya
Percentage: 91.0 %
Grade: A+
```

Figure 3. Class average, topper and ranking

8. Testing

Test	Input / Action	Expected result
Add valid student	Unique roll, name, marks 0–100	Record added and saved
Duplicate roll	Use an existing roll number	Duplicate rejected
Invalid marks	Text or value outside 0–100	Prompted to enter valid marks
View records	Choose option 2	All saved records displayed
Search	Existing / unknown roll	Record shown / not found message
Delete	Existing roll number	Record removed and file updated
Restart	Close and run again	Saved records loaded from JSON

Testing note: These are planned test cases and expected outcomes. Mark each test as passed only after executing it on your computer.

9. Concepts Used and Complexity

- Variables and expressions: totals, percentages and menu choice.
- Conditional statements: menu routing, input checks and grade assignment.
- Loops: repeated menu, mark entry and record traversal.
- Functions: modular program structure.
- Lists: ordered subject marks.
- Dictionaries: roll-number-based student records.
- Searching: dictionary lookup by roll number (average O(1)).
- Topper calculation: one pass through n records, O(n).
- Sorting: Python sorting, typically O(n log n).

10. Limitations and Future Scope

The current version is a small local command-line application. It does not include user accounts, multiple roles, a graphical interface or a database. Possible improvements include editing student details, exporting reports to CSV, adding subject-wise analysis, creating a GUI and using a database when the project requirements become more advanced.

11. Conclusion

The Student Management System demonstrates how basic Python programming constructs can solve a practical record-management problem. It combines data structures, functions, conditions, loops, searching, sorting and file handling in one understandable mini project. It also provides a foundation for future improvements as the student's programming skills develop.

References

1. Python Software Foundation, Python 3 Documentation: https://docs.python.org/3/
2. Python documentation for json: https://docs.python.org/3/library/json.html
3. Course notes for Introduction to Problem Solving and Programming.